

Max Flow: The Goldberg-Rao Algorithm

Andreas Panayi & Andreas Kalifommatos

May 2026

Combinatorial Optimization (Algorithms)

MSc: A.L.MA. - ΠΜΣ: A.Λ.MA.

National Kapodistrian University of Athens - National Technical University of Athens

The Maximum Flow Problem

The **Maximum Flow** problem is defined as follows.

- We are given a **directed graph** $G = (V, A)$, two distinguished nodes $s, t \in V$, and **integer capacities** $u(i, j) \geq 0$ for each arc $(i, j) \in A$.
- The goal is to find an **s - t flow** of **maximum value**.

Definition of an s - t Flow

An s - t flow is a function $f : A \rightarrow \mathbb{R}_{\geq 0}$ satisfying:

- **Capacity constraints:**

$$0 \leq f(i, j) \leq u(i, j) \text{ for each arc } (i, j) \in A.$$

- **Flow conservation:** for each node $i \in V \setminus \{s, t\}$,

$$\sum_{j: (i, j) \in A} f(i, j) - \sum_{j: (j, i) \in A} f(j, i) = 0.$$

The **value of the flow** is

$$v(f) := \sum_{j: (s, j) \in A} f(s, j) - \sum_{j: (j, s) \in A} f(j, s).$$

Preliminaries

Residual Graph

Given a directed graph $G = (V, A)$, capacities c and a flow f , the **residual graph** $G_f = (V, A_f)$ is defined as follows.

- For each arc $e = (u, v) \in A$:
 - add the **forward residual arc** (u, v) with capacity $c(e) - f(e)$,
 - add the **backward residual arc** (v, u) with capacity $f(e)$.
- Keep only arcs with **positive residual capacity**.

The residual graph describes how the current flow can be increased or rerouted.

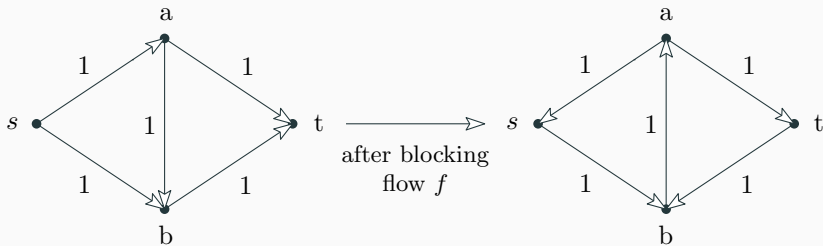
Blocking Flow

Let $G = (V, A)$ be a directed graph with capacities $u(i, j)$, source s , and sink t .

- A flow f is called a **blocking flow** if for every s - t path P in G , there exists an arc $(i, j) \in P$ such that $f(i, j) = u(i, j) > 0$.
- Equivalently, **every s - t path** in G contains a **saturated arc**.

Blocking flow $\nRightarrow$ Max flow | Max flow $\Rightarrow$ Blocking flow

Example



- blocking flow f : 1 unit along the (s,a,b,t) -path
- not a max flow

Blocking Flows and Dinitz Algorithm

Blocking Flows: Intuition

- The shortest augmenting path method removes one shortest s - t path at a time.
- Dinitz instead tries to eliminate all current shortest s - t paths at once via a *blocking flow*.
- After that, no shortest augmenting path remains, so the next augmenting path must be longer.
- Hence each phase makes clear global progress.

Dinitz Algorithm: Using Blocking Flows

Algorithm 1: Dinitz Algorithm

```
1  $f \leftarrow 0$ 
2 while there is still an augmenting path in  $G_f$  do
    // Compute the admissible graph  $G_A$ 
3   compute  $d(i) = \text{dist}_{G_f}(i, t)$  for all  $i \in V$ 
4    $\hat{A} \leftarrow \{(i, j) \in A_f \mid d(i) = d(j) + 1\}$ 
5    $\hat{u}(i, j) \leftarrow u_f(i, j)$  for all  $(i, j) \in \hat{A}$ 
6    $G_A \leftarrow (V, \hat{A})$  with capacities  $\hat{u}$ 
7
    // Compute a blocking flow in  $G_A$ 
8    $\hat{f} \leftarrow$  a blocking flow in  $G_A$ 
9
    // Update the current flow
10   $f \leftarrow f + \hat{f}$ 
11 return  $f$ 
```

Key Structural Facts

1. The algorithm always maintains a **feasible flow** f .
2. At every iteration, the **distance** from s to t **increases by at least 1**.
3. In each iteration, the algorithm computes a **blocking flow**; the details of this computation will be discussed later.

The s - t Distance in G_f Increases

- Let f, d be the current flow and distances, and f', d' those after the blocking-flow phase.

The s - t Distance in G_f Increases

- Let f, d be the current flow and distances, and f', d' those after the blocking-flow phase.
- Take a shortest s - t path P in $G_{f'}$. Then $d'(s) = |P|$.

The s - t Distance in G_f Increases

- Let f, d be the current flow and distances, and f', d' those after the blocking-flow phase.
- Take a shortest s - t path P in $G_{f'}$. Then $d'(s) = |P|$.
- Every arc $(i, j) \in P$ satisfies $d(i) \leq d(j) + 1$.

The s - t Distance in G_f Increases

- Let f, d be the current flow and distances, and f', d' those after the blocking-flow phase.
- Take a shortest s - t path P in $G_{f'}$. Then $d'(s) = |P|$.
- Every arc $(i, j) \in P$ satisfies $d(i) \leq d(j) + 1$.
- At least one arc of P was not admissible before; otherwise P would remain in the old admissible graph, contradicting the blocking flow.

The s - t Distance in G_f Increases

- Let f, d be the current flow and distances, and f', d' those after the blocking-flow phase.
- Take a shortest s - t path P in $G_{f'}$. Then $d'(s) = |P|$.
- Every arc $(i, j) \in P$ satisfies $d(i) \leq d(j) + 1$.
- At least one arc of P was not admissible before; otherwise P would remain in the old admissible graph, contradicting the blocking flow.
- Hence for at least one arc $(i, j) \in P$, we have $d(i) \leq d(j)$.

The s - t Distance in G_f Increases

- Let f, d be the current flow and distances, and f', d' those after the blocking-flow phase.
- Take a shortest s - t path P in $G_{f'}$. Then $d'(s) = |P|$.
- Every arc $(i, j) \in P$ satisfies $d(i) \leq d(j) + 1$.
- At least one arc of P was not admissible before; otherwise P would remain in the old admissible graph, contradicting the blocking flow.
- Hence for at least one arc $(i, j) \in P$, we have $d(i) \leq d(j)$.
- Therefore $d'(s) = |P| > \sum_{(i,j) \in P} (d(i) - d(j)) = d(s)$, so the distance increases by at least 1.

Blocking Flows in Unit-Capacity Graphs

Unit Capacity Case: Intuition

- In the unit capacity case, Dinitz behaves better than in the general case.
- The reason is that once the **shortest s - t paths** become longer, the remaining flow has fewer ways to pass through the graph.
- So after enough **blocking-flow phases**, only a **small amount of additional flow** can still remain.
- This leads to a better bound on the number of **phases**.

Unit Capacity Graphs and Running Time

A graph is called *unit capacity* if every arc has capacity either 0 or 1, that is, $u(i, j) \in \{0, 1\}$ for all $(i, j) \in A$.

- In this setting, the Dinitz algorithm performs only $O(\Lambda)$ iterations, where $\Lambda = \min\{\sqrt{m}, 2n^{2/3}\}$.
- Furthermore, each iteration, namely each blocking-flow computation, can be carried out in $O(m)$ time.

Therefore, in unit capacity graphs, the total running time is $O(\Lambda m)$.

Distance Levels and Associated Cuts

- For each integer $k \geq 0$, the *distance level* k is

$$B(k) = \{ i \in V \mid d(i) = k \}.$$

- Thus, $B(k)$ contains exactly the vertices whose distance to t is k .
- The *cut* determined by *distance level* k is

$$S(k) = \{ i \in V \mid d(i) \geq k \}.$$

- So $S(k)$ contains all vertices lying at level k or farther away from t .

Bounding the Number of Iterations by $O(\sqrt{m})$

- By structural fact 2, after $\sqrt{m}$ iterations we have $d(s) \geq \sqrt{m}$.

Bounding the Number of Iterations by $O(\sqrt{m})$

- By structural fact 2, after $\sqrt{m}$ iterations we have $d(s) \geq \sqrt{m}$.
- Hence there are at least $\sqrt{m}$ distinct nonempty distance levels, so we consider the cuts

$$S(1), S(2), \dots, S(\sqrt{m}).$$

Bounding the Number of Iterations by $O(\sqrt{m})$

- By structural fact 2, after $\sqrt{m}$ iterations we have $d(s) \geq \sqrt{m}$.
- Hence there are at least $\sqrt{m}$ distinct nonempty distance levels, so we consider the cuts

$$S(1), S(2), \dots, S(\sqrt{m}).$$

- Each arc can cross at most one of these cuts, since it can go between only one pair of consecutive levels.

Bounding the Number of Iterations by $O(\sqrt{m})$

- By structural fact 2, after $\sqrt{m}$ iterations we have $d(s) \geq \sqrt{m}$.
- Hence there are at least $\sqrt{m}$ distinct nonempty distance levels, so we consider the cuts

$$S(1), S(2), \dots, S(\sqrt{m}).$$

- Each arc can cross at most one of these cuts, since it can go between only one pair of consecutive levels.
- Therefore, one of these cuts must have at most $\sqrt{m}$ crossing arcs.

Bounding the Number of Iterations by $O(\sqrt{m})$

- By structural fact 2, after $\sqrt{m}$ iterations we have $d(s) \geq \sqrt{m}$.
- Hence there are at least $\sqrt{m}$ distinct nonempty distance levels, so we consider the cuts

$$S(1), S(2), \dots, S(\sqrt{m}).$$

- Each arc can cross at most one of these cuts, since it can go between only one pair of consecutive levels.
- Therefore, one of these cuts must have at most $\sqrt{m}$ crossing arcs.
- Since capacities are unit, this cut has residual capacity at most $\sqrt{m}$, so at most $\sqrt{m}$ more units of flow remain.

Bounding the Number of Iterations by $O(\sqrt{m})$

- By structural fact 2, after $\sqrt{m}$ iterations we have $d(s) \geq \sqrt{m}$.
- Hence there are at least $\sqrt{m}$ distinct nonempty distance levels, so we consider the cuts

$$S(1), S(2), \dots, S(\sqrt{m}).$$

- Each arc can cross at most one of these cuts, since it can go between only one pair of consecutive levels.
- Therefore, one of these cuts must have at most $\sqrt{m}$ crossing arcs.
- Since capacities are unit, this cut has residual capacity at most $\sqrt{m}$, so at most $\sqrt{m}$ more units of flow remain.
- Thus, after $\sqrt{m}$ iterations, at most $\sqrt{m}$ more iterations are possible, giving a total of $O(\sqrt{m})$.

Bounding the Number of Iterations by $O(n^{2/3})$

- By structural fact 2, after $2n^{2/3}$ iterations we have $d(s) \geq 2n^{2/3}$.

Bounding the Number of Iterations by $O(n^{2/3})$

- By structural fact 2, after $2n^{2/3}$ iterations we have $d(s) \geq 2n^{2/3}$.
- Hence there are at least $2n^{2/3}$ distinct nonempty distance levels.

Bounding the Number of Iterations by $O(n^{2/3})$

- By structural fact 2, after $2n^{2/3}$ iterations we have $d(s) \geq 2n^{2/3}$.
- Hence there are at least $2n^{2/3}$ distinct nonempty distance levels.
- Therefore, for some k , both consecutive levels satisfy

$$|B(k)| \leq n^{1/3} \quad \text{and} \quad |B(k-1)| \leq n^{1/3}.$$

Bounding the Number of Iterations by $O(n^{2/3})$

- By structural fact 2, after $2n^{2/3}$ iterations we have $d(s) \geq 2n^{2/3}$.
- Hence there are at least $2n^{2/3}$ distinct nonempty distance levels.
- Therefore, for some k , both consecutive levels satisfy

$$|B(k)| \leq n^{1/3} \quad \text{and} \quad |B(k-1)| \leq n^{1/3}.$$

- Any arc crossing the cut $S(k)$ must go from $B(k)$ to $B(k-1)$, so the number of such arcs is at most

$$|B(k)| |B(k-1)| \leq n^{2/3}.$$

Bounding the Number of Iterations by $O(n^{2/3})$

- By structural fact 2, after $2n^{2/3}$ iterations we have $d(s) \geq 2n^{2/3}$.
- Hence there are at least $2n^{2/3}$ distinct nonempty distance levels.
- Therefore, for some k , both consecutive levels satisfy

$$|B(k)| \leq n^{1/3} \quad \text{and} \quad |B(k-1)| \leq n^{1/3}.$$

- Any arc crossing the cut $S(k)$ must go from $B(k)$ to $B(k-1)$, so the number of such arcs is at most

$$|B(k)| |B(k-1)| \leq n^{2/3}.$$

- Since capacities are unit, this cut has residual capacity at most $n^{2/3}$, so at most $n^{2/3}$ more units of flow remain.

Bounding the Number of Iterations by $O(n^{2/3})$

- By structural fact 2, after $2n^{2/3}$ iterations we have $d(s) \geq 2n^{2/3}$.
- Hence there are at least $2n^{2/3}$ distinct nonempty distance levels.
- Therefore, for some k , both consecutive levels satisfy

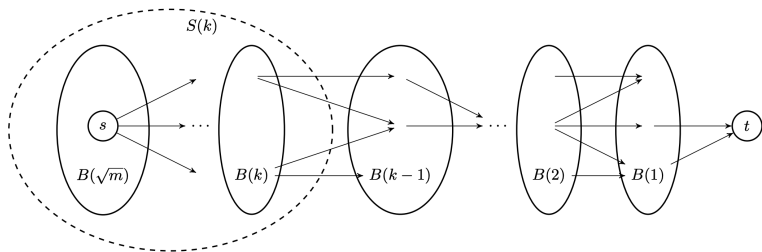
$$|B(k)| \leq n^{1/3} \quad \text{and} \quad |B(k-1)| \leq n^{1/3}.$$

- Any arc crossing the cut $S(k)$ must go from $B(k)$ to $B(k-1)$, so the number of such arcs is at most

$$|B(k)| |B(k-1)| \leq n^{2/3}.$$

- Since capacities are unit, this cut has residual capacity at most $n^{2/3}$, so at most $n^{2/3}$ more units of flow remain.
- Thus the total number of iterations is $O(n^{2/3})$.

Lemma Illustration



Computing a Blocking Flow in $O(m)$

- Repeatedly follow a path in the admissible graph G_A .
- If the path reaches t , augment by 1; since capacities are unit, every arc on the path becomes saturated and is deleted.
- If the path gets stuck, delete the last arc and backtrack.
- Thus, arcs are only deleted, never reused, and each arc is deleted at most once.

The Goldberg-Rao algorithm

- Maintain a **scale parameter** Δ so that arcs between consecutive distance levels have residual capacity at most Δ .
- After $\Lambda = \min\{\sqrt{m}, 2n^{2/3}\}$ **blocking-flow iterations**, this would give an s - t cut of residual capacity at most $\Lambda\Delta$.
- **Halving** Δ each time, and starting from an initial bound of at most nU , gives $O(\Lambda \log(nU))$ blocking-flow **computations**.
- To make this intuition **rigorous**, the algorithm introduces several additional ideas.

Lengths and Admissible Arcs

- Introduce new arc lengths: $\ell(i, j) = 1$ if $u_f(i, j) \leq \Delta$, and $\ell(i, j) = 0$ otherwise.
- Redefine $d(i)$ as the shortest distance from i to t with respect to these lengths.
- Then every residual arc satisfies $d(i) \leq d(j) + \ell(i, j)$.
- Hence, if an arc goes from level k to level $k - 1$, then necessarily $\ell(i, j) = 1$, so its residual capacity is at most Δ .
- The new **admissible arcs** are $\hat{A} = \{(i, j) \in A_f \mid d(i) = d(j) + \ell(i, j)\}$.

- An arc (i, j) is called **special** if:
 1. $d(i) = d(j)$,
 2. $\Delta/2 \leq u_f(i, j) \leq \Delta$,
 3. $u_f(j, i) > \Delta$.
- For a **special arc**, we reset its length to $\ell(i, j) = 0$.
- This does **not change the distances**, since already $d(i) = d(j)$.
- But it makes (i, j) **admissible**, because now $d(i) = d(j) + \ell(i, j)$.

A New Difficulty

- Using 0/1 lengths lets us argue that after Δ blocking-flow iterations, the residual minimum s - t cut has capacity at most $\Delta\Delta$.
- However, this change means that the previous blocking-flow analysis no longer goes through automatically.
- In particular, it is no longer clear that the distance from s to t must increase after each blocking-flow computation.

A Second Difficulty

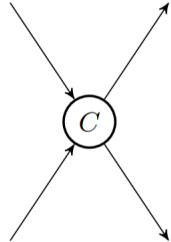
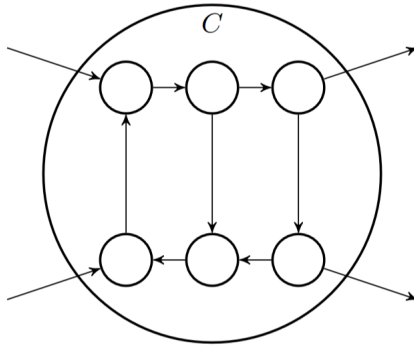
- In the previous algorithms, the **admissible graph** was **acyclic by construction**.
- All blocking-flow subroutines rely on this (naive $O(mn)$, dynamic-trees $O(m \log n)$, the size-bounded dynamic-trees $O(m \log(n^2/m))$).
- Now, since **length-0 admissible arcs** are allowed, the admissible graph may contain **cycles**.

Solving the Second Difficulty: Contraction

The admissible graph is layered: length-1 admissible arcs go to the next distance layer, while length-0 admissible arcs stay inside the same layer.

- Within each distance layer, compute the **strongly connected components** formed by the length-0 admissible arcs.
- Contract each such component into a **supernode**; this removes all directed cycles inside the layers.
- Compute the blocking flow on the **contracted admissible graph**.

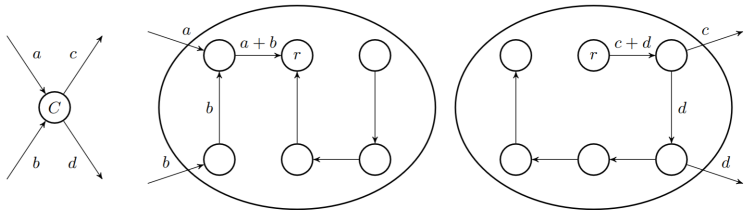
Contracting Strongly Connected Components



Solving the Second Difficulty: Lifting the Flow

- For each component, choose a **root** r .
- Since the component is strongly connected, choose an **intree** to r and an **outtree** from r .
- Route all incoming flow to r through the intree.
- Then route the same amount from r to the outgoing arcs through the outtree.
- By construction, the flow through a component is capped at $\Delta/4$, so overlapping tree arcs still have enough residual capacity.

Lifting the Flow



Goldberg-Rao Algorithm

Algorithm 2: Goldberg-Rao Algorithm

```
1  $f \leftarrow 0, F \leftarrow mU$ 
2 while  $F \geq 1$  do
3    $\Delta \leftarrow F/(2\Lambda)$ 
4   for  $5\Lambda$  iterations do
5      $\ell(i, j) \leftarrow 0$  if  $u_f(i, j) > \Delta$ , else  $\ell(i, j) \leftarrow 1$ 
6     Compute distances  $d(i)$  from  $i$  to  $t$  using lengths  $\ell$ 
7      $\hat{A} \leftarrow \{(i, j) \in A_f \mid d(i) = d(j) + \ell(i, j), u_f(i, j) > 0\}$ 
8     Add special arcs  $(i, j)$  to  $\hat{A}$ ;
9     Shrink Strongly Connected Components in  $\hat{A}$ 
10    Find either a flow  $\tilde{f}$  of value  $\Delta/4$  or a blocking flow  $\tilde{f}$ 
11    Compute  $\hat{f}$  by routing  $\tilde{f}$  in the uncontracted components
12     $f \leftarrow f + \hat{f}$ 
13   $F \leftarrow F/2$ 
14 return  $f$ 
```

Solving the First Difficulty: Weak Inequality

- It suffices to take a shortest s - t path P in $G_{f'}$ and show that every arc satisfies $d(i) \leq d(j) + \ell'(i, j)$, with strict inequality for at least one arc; then the sum telescopes and gives $d'(s) > d(s)$.

Solving the First Difficulty: Weak Inequality

- It suffices to take a shortest s - t path P in $G_{f'}$ and show that every arc satisfies $d(i) \leq d(j) + \ell'(i, j)$, with strict inequality for at least one arc; then the sum telescopes and gives $d'(s) > d(s)$.
- The Dinitz argument first gives the old-length inequality: $d(i) \leq d(j) + \ell(i, j)$, for every $(i, j) \in P$.

Solving the First Difficulty: Weak Inequality

- It suffices to take a shortest s - t path P in $G_{f'}$ and show that every arc satisfies $d(i) \leq d(j) + \ell'(i, j)$, with strict inequality for at least one arc; then the sum telescopes and gives $d'(s) > d(s)$.
- The Dinitz argument first gives the old-length inequality: $d(i) \leq d(j) + \ell(i, j)$, for every $(i, j) \in P$.
- The only new danger is that an arc becomes shorter, namely $d(i) = d(j) + 1$, $\ell(i, j) = 1$ and $\ell'(i, j) = 0$.

Solving the First Difficulty: Weak Inequality

- It suffices to take a shortest s - t path P in $G_{f'}$ and show that every arc satisfies $d(i) \leq d(j) + \ell'(i, j)$, with strict inequality for at least one arc; then the sum telescopes and gives $d'(s) > d(s)$.
- The Dinitz argument first gives the old-length inequality: $d(i) \leq d(j) + \ell(i, j)$, for every $(i, j) \in P$.
- The only new danger is that an arc becomes shorter, namely $d(i) = d(j) + 1$, $\ell(i, j) = 1$ and $\ell'(i, j) = 0$.
- Since $d(i) = d(j) + 1$, the reverse arc (j, i) was not admissible; hence the residual capacity of (i, j) cannot increase, contradicting $\ell'(i, j) = 0$.

Solving the First Difficulty: Strict Inequality I

- Since a blocking flow was computed, some arc $(i, j) \in P$ was not admissible before.

Solving the First Difficulty: Strict Inequality I

- Since a blocking flow was computed, some arc $(i, j) \in P$ was not admissible before.
- **Case 1:** suppose $u_f(i, j) = 0$.

Solving the First Difficulty: Strict Inequality I

- Since a blocking flow was computed, some arc $(i, j) \in P$ was not admissible before.
- **Case 1:** suppose $u_f(i, j) = 0$.
- Since (i, j) appears in $G_{f'}$, flow must have been pushed on the reverse arc (j, i) .

Solving the First Difficulty: Strict Inequality I

- Since a blocking flow was computed, some arc $(i, j) \in P$ was not admissible before.
- **Case 1:** suppose $u_f(i, j) = 0$.
- Since (i, j) appears in $G_{f'}$, flow must have been pushed on the reverse arc (j, i) .
- Thus, the only way strictness fails, i.e. $d(i) \not\leq d(j) + \ell'(i, j)$, is when $\ell'(i, j) = 0$, hence $u_{f'}(i, j) > \Delta$.

Solving the First Difficulty: Strict Inequality I

- Since a blocking flow was computed, some arc $(i, j) \in P$ was not admissible before.
- **Case 1:** suppose $u_f(i, j) = 0$.
- Since (i, j) appears in $G_{f'}$, flow must have been pushed on the reverse arc (j, i) .
- Thus, the only way strictness fails, i.e. $d(i) \not\leq d(j) + \ell'(i, j)$, is when $\ell'(i, j) = 0$, hence $u_{f'}(i, j) > \Delta$.
- This would require increasing $u_f(i, j)$ from 0 to more than Δ , impossible under the $\Delta/4$ cap.

Solving the First Difficulty: Strict Inequality I

- Since a blocking flow was computed, some arc $(i, j) \in P$ was not admissible before.
- **Case 1:** suppose $u_f(i, j) = 0$.
- Since (i, j) appears in $G_{f'}$, flow must have been pushed on the reverse arc (j, i) .
- Thus, the only way strictness fails, i.e. $d(i) \not\leq d(j) + \ell'(i, j)$, is when $\ell'(i, j) = 0$, hence $u_{f'}(i, j) > \Delta$.
- This would require increasing $u_f(i, j)$ from 0 to more than Δ , impossible under the $\Delta/4$ cap.
- Hence $d(i) < d(j) + \ell'(i, j)$.

Solving the First Difficulty: Strict Inequality II

- **Case 2:** suppose $u_f(i, j) > 0$, but (i, j) was not admissible.

Solving the First Difficulty: Strict Inequality II

- **Case 2:** suppose $u_f(i, j) > 0$, but (i, j) was not admissible.
- Then already $d(i) < d(j) + \ell(i, j)$.

Solving the First Difficulty: Strict Inequality II

- **Case 2:** suppose $u_f(i, j) > 0$, but (i, j) was not admissible.
- Then already $d(i) < d(j) + \ell(i, j)$.
- Strictness could fail only if $\ell(i, j) = 1$, $d(i) = d(j)$, and $\ell'(i, j) = 0$.

Solving the First Difficulty: Strict Inequality II

- **Case 2:** suppose $u_f(i, j) > 0$, but (i, j) was not admissible.
- Then already $d(i) < d(j) + \ell(i, j)$.
- Strictness could fail only if $\ell(i, j) = 1$, $d(i) = d(j)$, and $\ell'(i, j) = 0$.
- This is exactly the same-level danger handled by special arcs.

Solving the First Difficulty: Strict Inequality II

- **Case 2:** suppose $u_f(i, j) > 0$, but (i, j) was not admissible.
- Then already $d(i) < d(j) + \ell(i, j)$.
- Strictness could fail only if $\ell(i, j) = 1$, $d(i) = d(j)$, and $\ell'(i, j) = 0$.
- This is exactly the same-level danger handled by special arcs.
- Since (i, j) was not admissible, it was not made special; therefore $u_f(i, j) < \frac{\Delta}{2}$, making $\ell'(i, j) = 0$ would require too large a residual-capacity increase.

Solving the First Difficulty: Strict Inequality II

- **Case 2:** suppose $u_f(i, j) > 0$, but (i, j) was not admissible.
- Then already $d(i) < d(j) + \ell(i, j)$.
- Strictness could fail only if $\ell(i, j) = 1$, $d(i) = d(j)$, and $\ell'(i, j) = 0$.
- This is exactly the same-level danger handled by special arcs.
- Since (i, j) was not admissible, it was not made special; therefore $u_f(i, j) < \frac{\Delta}{2}$, making $\ell'(i, j) = 0$ would require too large a residual-capacity increase.
- The $\Delta/2$ capacity-change bound rules this out, so $d(i) < d(j) + \ell'(i, j)$.

Correctness: Outer-Loop Progress (1)

Goal. After setting $\Delta = F/(2\Lambda)$ and performing 5Λ inner computations, we want the remaining residual max-flow value to be at most $F/2$.

Correctness: Outer-Loop Progress (1)

Goal. After setting $\Delta = F/(2\Lambda)$ and performing 5Λ inner computations, we want the remaining residual max-flow value to be at most $F/2$.

Case 1: many augmentations.

- If at least 4Λ computations return value $\Delta/4$, then we add at least

$$4\Lambda \cdot \frac{\Delta}{4} = \Lambda\Delta = \frac{F}{2}$$

units of flow.

Case 2: many blocking flows.

- If fewer than 4Λ computations return value $\Delta/4$, then at least Λ computations are blocking-flow computations.
- These increase the distance enough for the layer argument to give an s - t cut crossed by at most Λ arcs.
- Each crossing arc has residual capacity at most Δ , so the cut has capacity at most

$$\Lambda\Delta = \frac{F}{2}.$$

Hence the current flow becomes at least $F/2$ closer to the maximum possible flow.

Goldberg-Rao Algorithm's Complexity

- Each outer iteration performs 5Λ blocking-flow computations.
- Each such computation costs $O(m \log n)$ using **dynamic trees**.
- The parameter F starts at mU and is halved after every outer iteration.
- Thus there are $O(\log(mU))$ outer iterations.
- Therefore,

$$\begin{aligned}\text{Total time} &= O(\Lambda m \log n \log(mU)) \\ &= O(\min\{\sqrt{m}, 2n^{2/3}\} m \log n \log(mU)).\end{aligned}$$

$$O(\min\{\sqrt{m}, 2n^{2/3}\} m \log n \log(mU))$$

$$O(\min\{\sqrt{m}, 2n^{2/3}\} m \log n \log(mU))$$

↓ **start with $F = nU$ instead of $F = mU$**

$$O(\min\{\sqrt{m}, 2n^{2/3}\} m \log n \log(nU))$$

$$O(\min\{\sqrt{m}, 2n^{2/3}\} m \log n \log(mU))$$

↓ start with $F = nU$ instead of $F = mU$

$$O(\min\{\sqrt{m}, 2n^{2/3}\} m \log n \log(nU))$$

↓ use size-bounded dynamic trees

$$O(\min\{\sqrt{m}, 2n^{2/3}\} m \log(n^2/m) \log(nU))$$

Improving the Complexity

$$O(\min\{\sqrt{m}, 2n^{2/3}\} m \log n \log(mU))$$

↓ start with $F = nU$ instead of $F = mU$

$$O(\min\{\sqrt{m}, 2n^{2/3}\} m \log n \log(nU))$$

↓ use size-bounded dynamic trees

$$O(\min\{\sqrt{m}, 2n^{2/3}\} m \log(n^2/m) \log(nU))$$

↓ more careful analysis

$$O(\min\{\sqrt{m}, 2n^{2/3}\} m \log(n^2/m) \log(U))$$

Computing a Blocking Flow

In graphs with no cycles, a **blocking flow** can be computed using:

- **Naive algorithm:** $O(nm)$,
- **Dynamic trees:** $O(m \log n)$,
- **Size-bounded dynamic trees:** $O(m \log(n^2/m))$.

Dynamic Trees for Blocking Flows

Maintain a **forest** F of admissible, unsaturated edges.

operation	meaning	time
<code>makeTree(v)</code>	initializes singleton tree	$O(1)$
<code>findRoot(v)</code>	find root of v 's tree	$O(\log n)$
<code>link(v, w, c)</code>	add (v, w) with capacity c	$O(\log n)$
<code>cut(v)</code>	remove v 's parent edge	$O(\log n)$
<code>findMin(v)</code>	bottleneck on v -to-root path	$O(\log n)$
<code>subtract(v, x)</code>	subtract x on v -to-root path	$O(\log n)$

Key point: **path bottlenecks** and **path updates** take **logarithmic time**.

High-Level Blocking-Flow Computation

Algorithm 3: Blocking Flow Computation using Dynamic Trees

```
1 Initialize every vertex as a singleton dynamic tree
2 while true do
3    $u \leftarrow$  root of  $s$ 
4   if  $u = t$  then
5     Find the bottleneck on the represented  $s$ - $t$  path
6     Subtract the bottleneck from all edges of this path
7     Cut and delete all zero-capacity edges
8   else if  $u$  has an outgoing admissible edge  $(u, w)$  then
9     Add edge  $(u, w)$  with capacity  $c(u, w)$            // advance
10  else                                           // we are at a dead end
11    if  $u = s$  then
12      break                                     //  $s$  is the dead end
13    else
14      Cut the tree edges entering  $u$ 
15      Delete them from the admissible graph

16 Return the flow as original capacities minus remaining capacities
```

Running Time

- Each loop iteration is one of three cases:
 - **advance**: link one new admissible edge;
 - **reached t** : update one represented path and remove zero-capacity edges;
 - **dead end**: cut forest edges entering u and delete them from the graph.
- Each edge is linked at most once and deleted at most once.
- Thus the algorithm performs $O(m)$ **dynamic-tree operations**.
- Since each operation costs $O(\log n)$, one blocking flow is computed in $O(m \log n)$.
- With **size-bounded dynamic trees**, this becomes $O(m \log(n^2/m))$.

Bibliography

References

- [GR98] Andrew V. Goldberg and Satish Rao. “**Beyond the Flow Decomposition Barrier**”. In: *Journal of the ACM* 45.5 (1998), pp. 783–797. DOI: [10.1145/290179.290181](https://doi.org/10.1145/290179.290181).
- [Wil19] David P. Williamson. ***Network Flow Algorithms***. Cambridge University Press, 2019. DOI: [10.1017/9781316888568](https://doi.org/10.1017/9781316888568).